

Data Structures (ETCCDS202)

Unit-4 Assignment: Non-Linear & Advanced Data Structures

Course Details

Name of the School:	SOET
Program (Class / Semester / Batch):	B.Tech (CSE / AIML)
Course Title:	Data Structures
Course Code:	ETCCDS202
Unit Title:	Non-Linear & Advanced Data Structures
Student Name:	Vansh Negi
Roll No.:	2501730158
Section:	A

Submission Date: 16 April 2026

Part A: Binary Search Tree (BST)

What is a Binary Search Tree?

A Binary Search Tree (BST) is a non-linear data structure in which each node contains a value and at most two children. It follows a specific property:

- All elements in the left subtree are smaller than the root
- All elements in the right subtree are greater than the root

This property makes searching, insertion, and deletion efficient compared to linear data structures.

Why inorder traversal prints sorted output?

Inorder traversal follows the sequence:

Left → Root → Right

Since smaller elements are always placed in the left subtree and larger elements in the right subtree, visiting nodes in this order results in elements being printed in ascending sorted order.

BST Operations

Insertion

Insertion places a new element at the correct position such that the BST property is maintained. The algorithm compares the value with the root and moves left or right accordingly.

Search

Search operation checks whether a value exists in the tree by comparing and traversing left or right.

Deletion

Deletion in BST involves three cases:

1. No child (Leaf node)

The node is simply removed.

2. One child

The node is replaced by its child.

3. Two children

The node is replaced by its inorder successor (smallest value in right subtree).

Time Complexity

Average Case:

$O(\log n)$ because the tree remains balanced and height is small.

Worst Case:

$O(n)$ when the tree becomes skewed (like a linked list).

Part B: Graph Traversal (BFS & DFS)

Graph Representation

A graph consists of nodes (vertices) and edges connecting them. In this assignment, the graph is represented using an adjacency list, where each node stores its neighbours along with edge weights.

Breadth First Search (BFS)

BFS explores nodes level by level starting from a given node.

- It uses a queue data structure
- It first visits all neighbours of a node before moving deeper

Working Principle:

1. Start from the source node
2. Visit all adjacent nodes
3. Continue level by level

Use Case:

Finding the shortest path in an unweighted graph.

Depth First Search (DFS)

DFS explores as deep as possible before backtracking.

- It uses recursion or stack
- It follows one path completely before exploring others

Working Principle:

1. Start from the source node
2. Move to one neighbour
3. Continue until no further nodes are available
4. Backtrack and explore remaining nodes

Use Case:

Cycle detection, path finding, and solving puzzles.

Difference Between BFS and DFS

Feature	BFS	DFS
Approach	Level-wise traversal	Depth-wise traversal
Data Structure	Queue	Stack/Recursion
Speed	Slower in deep graphs	Faster in deep paths
Use Case	Shortest path	Cycle detection

Part C: Hash Table (Separate Chaining)

What is Hashing?

Hashing is a technique used to map keys to indices in a fixed-size table using a hash function.

Example:

$\text{index} = \text{key} \% \text{table_size}$

This allows fast access, insertion, and deletion of data.

What is a Collision?

A collision occurs when two or more keys map to the same index.

Example:

If table size = 5

Keys: 10, 15, 20

All map to index 0

Separate Chaining

Separate chaining is a technique used to handle collisions. In this method, each index of the hash table stores a list of elements.

When multiple keys map to the same index, they are stored in the same list (chain).

Operations in Hash Table

Insert

Adds a key-value pair to the appropriate index.

Get

Searches for a key in the chain and returns its value.

Delete

Removes a key from the chain.

Time Complexity

Average Case:

$O(1)$ due to direct access using hash function.

Worst Case:

$O(n)$ when many elements fall into the same bucket.

Part D: Real-World Applications

Binary Search Tree

BST is used in databases for indexing and maintaining sorted data efficiently.

Graphs

Graphs are widely used in:

- Navigation systems (finding routes)
- Social networks (connections between users)
- Network routing

Hash Tables

Hash tables are used in:

- Caching systems
- Dictionaries and maps
- Fast data lookup systems

Part E: Short Reflection

This assignment helped in understanding how non-linear data structures work in real-world applications.

Binary Search Trees provide efficient searching and sorting. Graphs help in modeling networks and relationships, while hashing enables very fast data retrieval.

These structures are essential in modern software systems such as databases, search engines, and networking applications.

Through this assignment, the concepts of traversal, searching, and collision handling became much clearer and practical.